Solution: Longest Increasing Path in a Matrix

Problem Statement

Given a 2D matrix of size `m x n`, return the length of the `longest increasing path` in the matrix. The increasing path should have each element should be greater than the previous element in the path.

It is given that you can move only in 4-directions: `left`, `right`, `top`, and `bottom`.

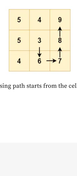
Examples

Example 1:

- **Input**: matrix =

```
[[3, 4, 5],
 [3, 2, 6],
 [2, 2, 1]]
```

- **Expected Output**: 4



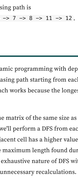
- **Justification**: The longest increasing path is `3 -> 4 -> 5 -> 6`. No other path has a greater length.

Example 2:

- **Input**: matrix =

```
[[5,4,9],
 [5,3,8],
 [4,6,7]]
```

- **Expected Output**: 5



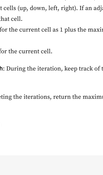
- **Justification**: The longest increasing path starts from the cell with value `3` (`1, 1`), moving to `6`, then `7`, `8`, and finally `9`.

Example 3:

- **Input**: matrix =

```
[[1, 2, 3, 4],
 [6, 5, 4, 3],
 [7, 8, 9, 2],
 [10, 11, 12, 1]]
```

- **Expected Output**: `10`



- **Justification**: The longest increasing path is `1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 11 -> 12`.

Solution

To solve this problem, we'll use dynamic programming with depth-first search (DFS). The idea is to store the length of the longest increasing path starting from each cell, so we do not need to recompute it each time. This approach works because the longest path from any given cell is fixed once calculated.

In the beginning, we'll create a cache matrix of the same size as the input matrix to store the lengths of the longest paths. Then, we'll perform a DFS from each cell, checking its four adjacent cells (up, down, left, right). If an adjacent cell has a higher value, we'll continue the DFS from that cell. We'll update the cache with the maximum length found during these searches. This approach is efficient because it combines the exhaustive nature of DFS with the optimization of dynamic programming, allowing us to avoid unnecessary recalculations.

Step-by-step Algorithm

1. **Initialize Cache**: Create a cache matrix of the same size as the input matrix. Initialize all values to 0. This cache will store the length of the longest increasing path starting from each cell.
2. **Iterate Through Matrix**: Go through each cell in the input matrix. For each cell, perform a Depth-First Search (DFS) if its corresponding value in the cache is 0.
3. **Depth-First Search (DFS)**:
   - If the current cell is out of bounds or its value is not greater than the previous cell's value, return 0.
   - If the cache value at the current cell is non-zero, return it (to avoid recomputation).
   - Explore all four adjacent cells (up, down, left, right). If an adjacent cell has a greater value, recursively call DFS on that cell.
   - Update the cache value for the current cell as 1 plus the maximum length obtained from the adjacent cells.
   - Return the cache value for the current cell.
4. **Track Maximum Path Length**: During the iteration, keep track of the maximum length found across all cells.
5. **Return Result**: After completing the iterations, return the maximum path length found.

Algorithm Walkthrough

Let's use the matrix:

```
[[5, 4, 9],
 [5, 3, 8],
 [4, 6, 7]]
```

Step 1: Start DFS at (0, 0) - Value 5

- Initial exploration. There are not any adjacent values greater than 5.
- Cache matrix after update:

```
1 0 0
0 0 0
0 0 0
```

- maxPath: `1`

Step 2: Explore (0, 1) - Value 4

- Finds a path to 5, updates its path length to 2.
- Cache matrix:

```
1 2 0
0 0 0
0 0 0
```

- maxPath: `2`

Step 3: Move to (0, 2) - Value 9

- Directly reaches the end of a potential path.
- Cache matrix update reflects this cell as an endpoint of a path.

```
1 2 1
0 0 0
0 0 0
```

- maxPath: `2`

Step 4: Explore (1, 0) - Value 5

- Same value as (0,0), no increase.
- Cache matrix:

```
1 2 1
1 0 0
0 0 0
```

- maxPath: `2`

Step 5: Explore from (1, 1) - Value 3

- Moving to the top: Find a path from 3 to 4, update `maxLength` to `1 + 2 = 3`.
- Moving to the bottom: Find a path from 3 to 6.
- Next, explore the path from 6 to 7, as it is the only valid move.
- Next, explore the path from 7 to 8, as it is the only valid move.
- Next, explore the path from 8 to 9, as it is the only valid move.
- After exploring, the 8 to 9 path, the updated cache matrix is:

```
1 2 1
1 0 2
0 0 0
```

- After exploring, the 7 to 8 path, the updated cache matrix is:

```
1 2 1
1 0 2
0 0 3
```

- After exploring, the 6 to 7 path, the updated cache matrix is:

```
1 2 1
1 0 2
0 4 3
```

- After exploring, the 3 to 6 path, the updated cache matrix is:

```
1 2 1
1 5 2
0 4 3
```

- maxPath: `5`.
- Continue traversing each cell of the matrix this way. The length of the maximum increasing path would be 5 and path would be either `3 -> 6 -> 7 -> 8 -> 9` or `4 -> 6 -> 7 -> 8 -> 9`

Code

Java

```
public class Solution {
    private int[][] matrix; // The input matrix
    private int[][] cache; // Cache to store lengths of longest increasing paths
    private int rows, cols; // Dimensions of the matrix

    // Main method to find the longest increasing path
    public int longestIncreasingPath(int[][] matrix) {
        // Edge case check
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return 0;
        }

        this.matrix = matrix;
        rows = matrix.length;
        cols = matrix[0].length;
        cache = new int[rows][cols];
        int maxPath = 0;

        // Iterate through each cell of the matrix
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

- **Time Complexity**: `O(M * N)`, where `M` and `N` are the dimensions of the matrix. This is because each cell in the matrix is the starting point for DFS only once, thanks to caching.
- **Space Complexity**: `O(M * N)`, due to the cache matrix and recursion stack. The cache stores the longest path starting from each cell, and the recursion stack depth can potentially be as large as the number of cells in the matrix in the worst case.